Lecture 10: Control and execution

Lui Holder-Pearson

September 24, 2026

Learning Goals

- Understand the *fetch*, *decode*, and *execute* states.
- Understand how *instructions* are stored within the ATMega32u2.
- Be able to interpret basic AVR instructions.
- Understand how the program counter is used and manipulated for programme flow.
- Be able to interpret some basic examples of assembly language.

Further Reading

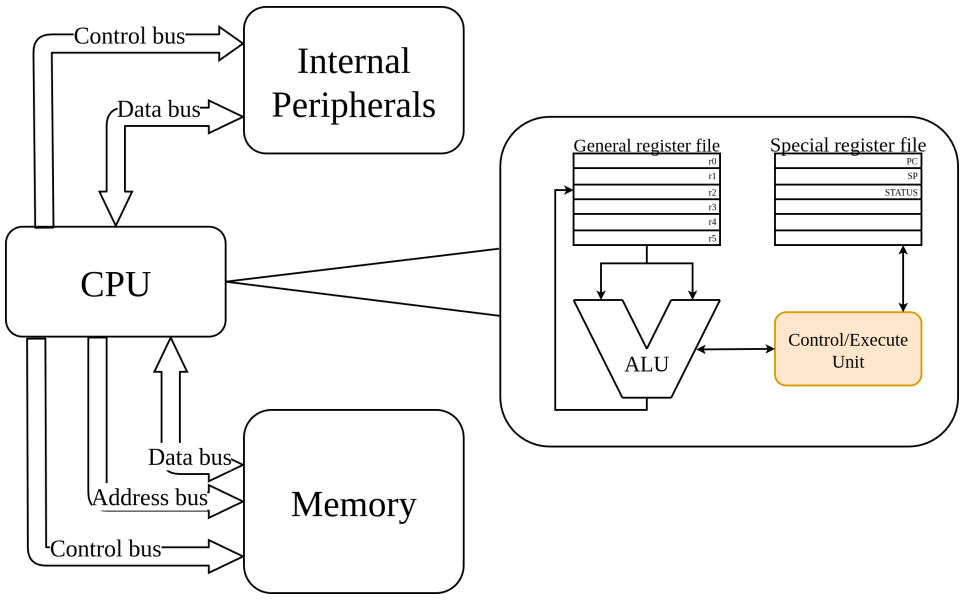
Tom Scott video discussing the fetch-execute cycle.
AVR instruction set (direct link to pdf).

References

Ciaran Moore's past ENCE260 lecture notes
Rizzoni and Kearns. *Principals and Applications of Electrical Engineering*, Chapters 13 & 14.

The whole picture

We should now be familiar with the entire contents of the basic computer architecture diagram except the control/execute unit:



1 Fetch, decode, execute

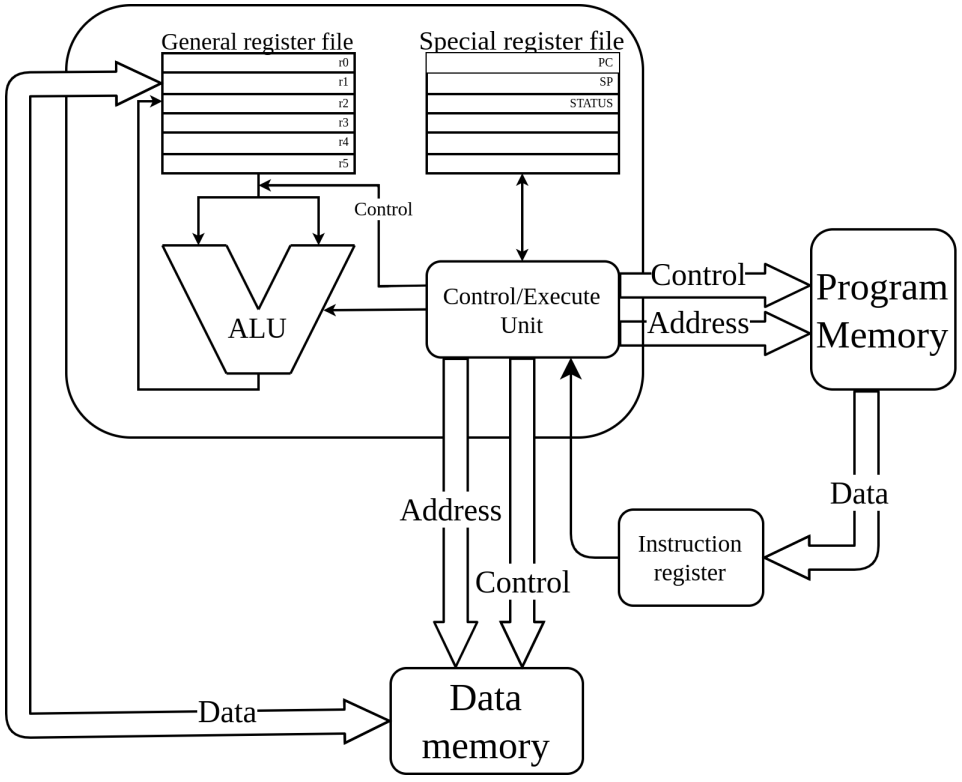
We have put all of the systems in place to be able to perform various actions on a microcontroller, but how does it actually *get done*?

The control/execute unit is a finite state machine, which actually enacts the *instruction*. This leaves some large unanswered questions:

- What does the control unit FSM look like?
- What are its states?
- What are its inputs and outputs?

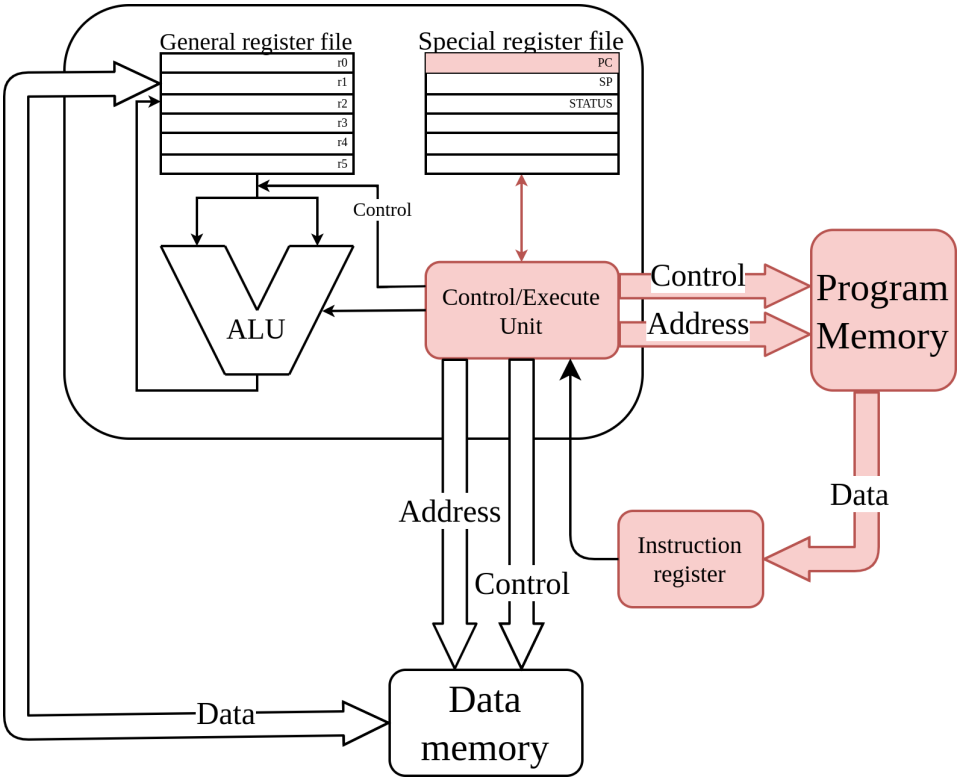
More specifically, the interactions that are going on are:

Recall: Harvard architecture (the most common by far) has a separate data bus for the data from memory and the program data.



1.1 Fetch

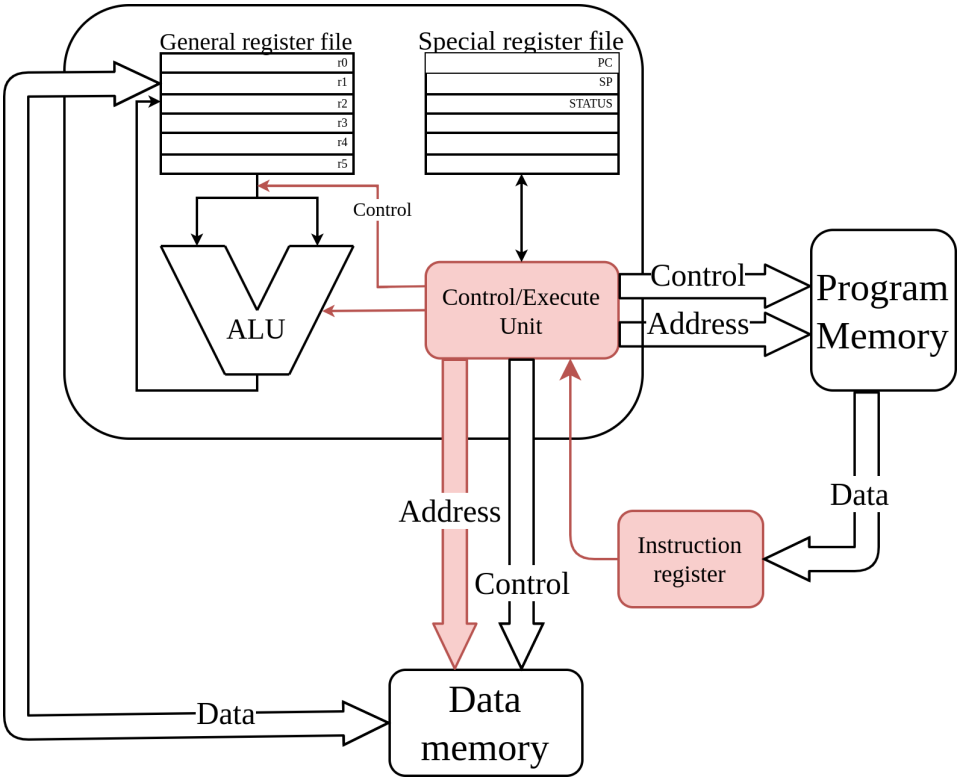
To perform an instruction, the instruction must actually be fetched. This means to load it from memory.



- PC:
- Program memory:
- IR:

1.2 Decode

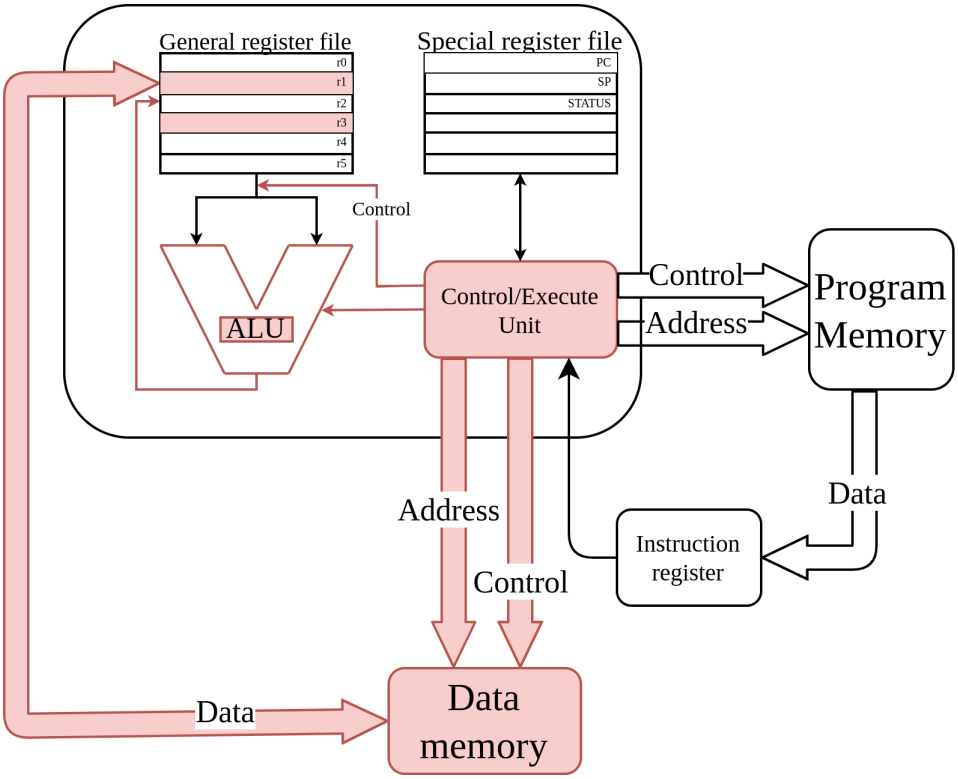
Decoding is the setting up for the execution. This is the interpretation of the *opcode* within the instruction register



- IR:
- C/E:
- ALU:
- Buses:

1.3 Execute

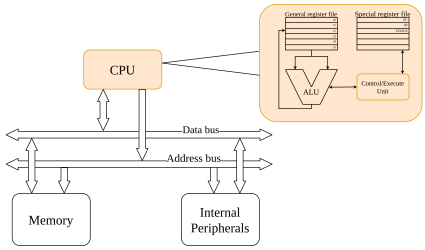
Finally we have reached the crux of ENCE260: Actually performing an instruction!



- GPR:
- ALU:
- Memory:

Thus, as a FSM:

2 Instructions



Broadly, there are three types of instruction:

1. Data transfer

2. Arithmetic/logic

3. Branching

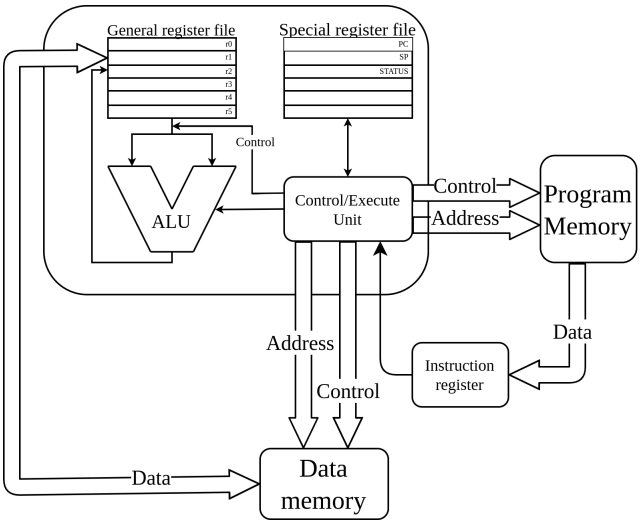
Collectively, the instructions form an instruction set, and the way that this set is designed is called the *instruction set architecture* (ISA).

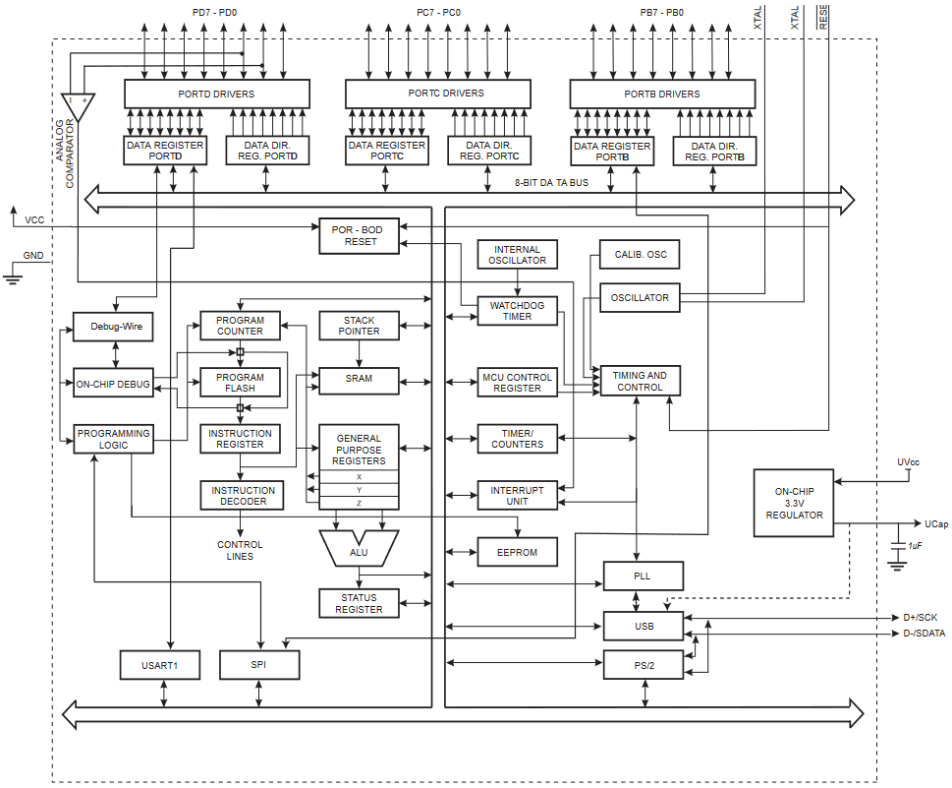
There are two distinct types of ISAs:

1. CISC:
   - ALU operations access registers or memory
   - Variable length of instruction
   - Variable number of clock cycles to complete instructions
2. RISC:
   - ALU operations access only registers
   - Separate data transfer functions
   - One clock cycler per instruction

2.1 ATMega32u2

- 8-bit microcontroller used in the UCFunKit.
- Atmel AVR Instruction Set: RISC architecture, with all instructions 16-bits long.
- Harvard architecture





2.2 AVR instructions

The instruction itself is a sequence of 1s and 0s which contains the necessary information to perform the operation. The required information depends on the type of operation. For example:

Data transfer operation

$$Memory \rightarrow Registers$$

We can pack all of the information required into a clean table:

Syntax	Operands	Operation
Control registers	Opcode	Flags

Addition operation

$$a + b$$

Syntax	Operands	Operation
Control registers	Opcode	Flags

Status register (SREG)	
C	Carry Flag
Z	Zero Flag
N	Negative Flag
V	Two's complement overflow indicator
S	N $\oplus$ V, for signed tests
H	Half Carry Flag
T	Transfer bit used by BLD and BST instructions
I	Global Interrupt Enable/Disable Flag
Registers and Operands	
Rd:	Destination (and source) register in the Register File
Rr:	Source register in the Register File
R:	Result after instruction is executed
K:	Constant data
k:	Constant address
X,Y,Z:	Indirect Address Register (X=R27:R26, Y=R29:R28, and Z=R31:R30)

Table 1: `add` opcode summary: Add instruction.

Syntax	Operands	Operation
`add Rd, Rr`	$0 \leq d \leq 31$, $0 \leq r \leq 31$	Rd $\leftarrow$ (Rd+Rr)
Control registers	Opcode	Flags
PC $\leftarrow$ PC + 1	0000 11rd dddd rrrr	H, S, V, N, Z, C

Table 2: `ldd` opcode summary: Load indirect from data space to register with displacement.

Syntax	Operands	Operation
`ldd Rd, Y+q`	$0 \leq d \leq 31$,$0 \leq q \leq 63$	Rd $\leftarrow$ (Y+q)
Control registers	Opcode	Flags
PC $\leftarrow$ PC + 1	01q0 qq0d dddd 1qqq	None

Table 3: `adiw` opcode summary: Add immediate to word.

Syntax	Operands	Operation
`adiw Rd+1:Rd,k`	d $\in \{24, 26, 28, 30\}$,$0 \leq k \leq 63$	Rd+1:Rd $\leftarrow$ Rd+1:Rd + k
Control registers	Opcode	Flags
PC $\leftarrow$ PC + 1	1001 0110 kkdd kkkk	S, V, N, Z, C

Table 4: `breq` opcode summary: Branch if equal.

Syntax	Operands	Operation
`breq k`	$-64 \leq k \leq 63$	Is result of CP, CPI, SUB, or SUBI = 0?
Control registers	Opcode	Flags
PC $\leftarrow$ PC + 1 (if Z = 0) PC $\leftarrow$ PC + k + 1 (if Z = 1)	1111 00kk kkkk k001	None

Some key points:

- Instructions are written as `name dest, source`
- All instructions can be written as 16-bit opcode
- Instructions may modify flags

.

2.2.1 Addressing modes

1. Immediate

2. Direct

3. Register

4. Indirect

3 Example

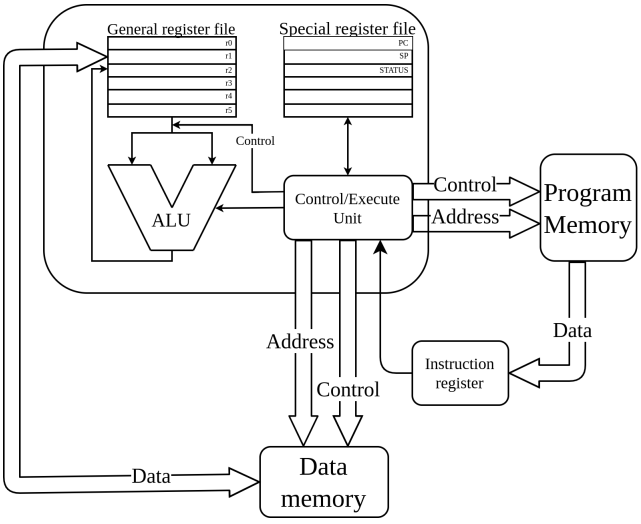
```c
int main(void) {
  int index;
  int sum;
  int array[4] = {-1, 3, 5, -2};
  sum = 0;
  for (index = 0; index < 4; index++) {
    sum = sum + array[index];
  }
  return 0;
}
```

Assembly:

```
// index++
ldd r24, Y+1   // Load the value of 'index' from
ldd r25, Y+2   // memory into r24 and r25
adiw r24, 0x01 // Add 0x01 to r25:r24

// Store r25:24 to memory (not shown here)

// Test for index >= 4
sbiw r24, 0x04 // Subtract 0x04 from r25:r24
brlt .-56      // Go back 56 instructions
               // if result was negative
```


General register file
r0
r1
r2
r3
r4
r5
Special register file
PC
SP
STATUS
Control
ALU
Control/Execute Unit
Control
Address
Program Memory
Data
Address
Instruction register
Control
Data
Data memory

Summary

- Understand the *fetch*, *decode*, and *execute* states.
  → These are the three states of the control and execute FSM, where the specific state transitions are governed by a clock and the overall programme flow.
  → Fetch retrieves an instruction from the programme memory,
  → Decode informs the control and execute unit how to configure the rest of the CPU, and the address and control buses.
  → Execute is when the instruction is carried out, and the result stored in the appropriate location.
- Understand how *instructions* are stored within the ATMega32u2.
  → Instructions are stored within 16-bit opcode instructions. Each opcode contains the specific sequence of `1`s and `0`s to specify the operation, as well as additional function parameters, such as memory/register addresses, and/or constants.
- Be able to interpret basic AVR instructions.
  → Similar to the examples we saw: `ldd`, `adiw`, `breq`.
- Understand how the program counter is used and manipulated for programme flow.
  → The program counter (PC) is a very special pointer which contains the location in memory of the next instruction to be read.
  → It is manipulated with branching `brXX` instructions, otherwise it is typically incremented by one every instruction.
- Be able to interpret some basic examples of assembly language.

Next Lecture: Lecture 11 - Code and function calls

4 Questions?